

Workshop 2: Lissajous curves

In this workshop you will use OpenMP to parallelise a program which calculates Lissajous curves. In order to parallelise the program correctly you will need to consider variable scoping and whether there are any loop-carried dependencies.

The workshop is designed to be carried out using the University's HPC facilities ISCA. If you are not on campus, you will need to use VPN before connecting to ISCA. On the module ELE page there are instructions describing how to log in to ISCA remotely under the "Workshop 2" section.

1. Getting set up

This section describes how to log in to ISCA and obtain the example programs.

- Connect to ISCA using ssh – see instructions on 'Remote Access to ISCA HPC' under Week 2 Workshop.

```
ssh abc123@login.isca.ex.ac.uk
```

- The files you will need for Week 2 Workshops are available as a Tar file in Week 2 – 'Workshop 2 tar file'. Download this file to your local computer.
- You can make a directory for the workshop activities in ISCA. You will need to log in to ISCA to do this.

```
mkdir <Name of the Directory>
```

- Upload the tar file to ISCA. You will have to log in to your local computer terminal. Define the path correctly in your local computer; abc123 is your university user name.

```
scp path/local/computer/workshop2.tar  
abc123@login.isca.ex.ac.uk:~FOLDERinISCA
```

- You will see something below if your upload is successful. Now you can log back to ISCA and navigate to your working directory.



A terminal window showing the process of logging into ISCA and uploading a file. The prompt is (jp1143@login.isca.ex.ac.uk) Password:. The user enters workshop2.tar. The prompt changes to PS C:\Users\jp1143> |. On the right side of the terminal, there is a progress bar showing 100% 3551 138.7KB/s 00:00.

- Unpack the tar file by running the following command:

```
tar xvf workshop2.tar
```

This will create a directory called `workshop2` which contains the example programs.

2. Lissajous curves

Lissajous curves are defined by the equations:

$$x = A \sin(\omega_x t + \delta_x) \quad y = B \sin(\omega_y t + \delta_y) \quad (1)$$

where ω_x , ω_y , δ_x and δ_y are constant parameters. These are parametric equations where the coordinates x and y are defined in terms of a parameter t . If ω_x/ω_y is rational (i.e. it can be written as a fraction) then x and y will describe a closed curve. Some example Lissajous curves are shown in figure 1

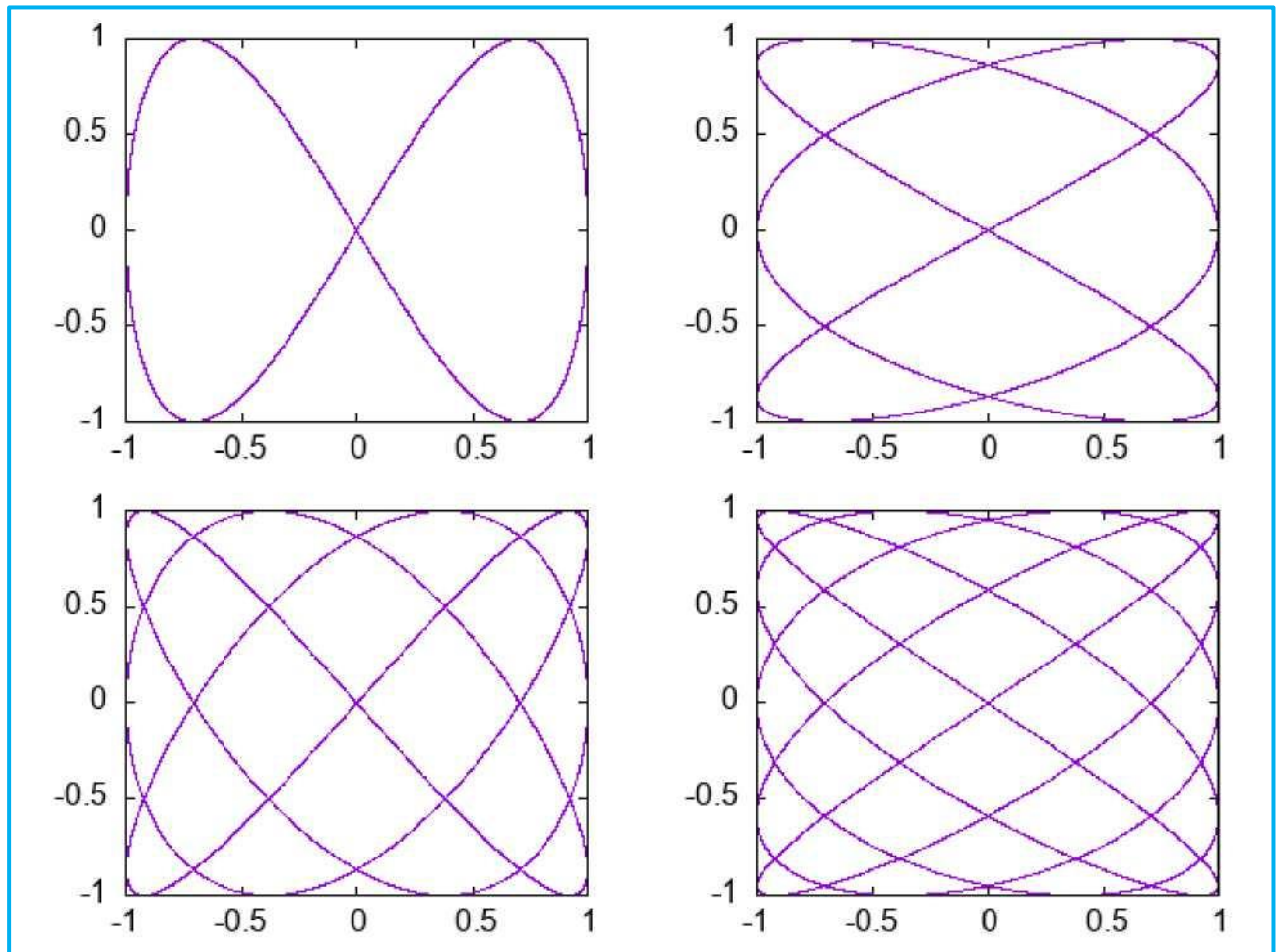


Figure 1: Example Lissajous curves: Top left $\omega_x = 1$, $\omega_y = 2$, $\delta_x = \pi/2$; top right $\omega_x = 3$, $\omega_y = 2$, $\delta_x = \pi/2$; lower left $\omega_x = 3$, $\omega_y = 4$, $\delta_x = \pi/4$; lower right $\omega_x = 5$, $\omega_y = 4$, $\delta_x = \pi/4$ ($\delta_y = 0$ for all curves).

3. The serial program

- Start by changing into the `workshop2` directory, and compile and run the example program `lissajous.c`:

```
cd workshop2
gcc -o lissajou -std=c99 lissajous.c -lm
./lissajou
```

When you run the program it will write a file called `output.dat` which contains the data for a Lissajous curve with the parameters $\omega_x = 1$, $\omega_y = 2$, $\delta_x = \pi/2$, $\delta_y = 0$.

- You can now plot the Lissajous curve using the gnuplot script provided below.

Run the below commands to load the necessary modules. You will be using the gnuplot tool:

```
module unuse /lustre/shared/modules/all
module use /lustre/shared/isca_compute/modules/all
module spider gnuplot
module add gnuplot
```

To plot the curve run the command:

```
gnuplot plot_lissajous
```

this will create a file in PNG format called `lissajous.png`.

- To view the file copy it to your local computer by running the following `scp` command on your local computer: To do this you will need to log in to your local computer terminal again (use `exit` command to log out from ISCA).

```
scp
abc123@login.isca.ex.ac.uk:/lustre/home/abc123/path/ISCAfolder/lissajous.png
.\path\to\local\computer\
```

where you should replace `abc123` with your university username.

You can also use MobaXTerm or WinSCP to download the png file.

You can now open the file with an appropriate PNG viewer on your local device.

- You can now log back into ISCA and navigate to your working directory. The file `output.dat` is your known good output which you can use later to test that your parallel version of the program is working correctly. To prevent this file being overwritten by later versions of the program rename the file:

```
mv output.dat output_kgo.dat
```

4. Parallelising the program

You are now ready to parallelise the program using OpenMP.

- You will need to choose a text editor to edit the program. The `nano`, `emacs` and `vi` editors are available on the blue room PCs computers. If you are not familiar with any of these editors I recommend using `nano`. If you get stuck in `vi` type `!q` to quit.

- Open the program in your chosen text editor and find the `for` loop in the `main` function.

Question 1: Are there any loop carried dependencies in this loop which prevent it being parallelised?

- There is also a `for` loop in the `write_to_file` function.

Question 2: Can the `for` loop in the `write_to_file` function be parallelised?

- Now parallelise the program using OpenMP making sure that the variables are correctly scoped. You should check that the results are correct by comparing against the known good output. You can use the `diff` command to check that the output matches the known good output from the serial version:

```
diff --brief output.dat output_kgo.dat
```

If there is no output from the `diff` command then the files are identical.

Question 3: How did you scope the variables to make sure the parallel version gives identical output to the serial version?

5. Generating multiple curves

The `workshop2` directory contains another version of the program called `lissajous_multi.c`. This version of the program calculates four Lissajous curves for the parameters shown in figure 1. There is also a plotting script called `plot_lissajous_multiple` for plotting the results.

- Open the `lissajous_multi.c` program in a text editor and find the part of the `main` function where the Lissajous curves are calculated. This calculation is now carried out by two nested `for` loops: an inner loop over `N_POINTS` points and an outer loop over `N_CURVES` curves.

Question 4: Which loop would you choose to parallelise and why?

- OpenMP provides a clause called `collapse` which is used to combine loops so that more than one loop can be parallelised with a single OpenMP pragma. Section 2.9.2 of the OpenMP 5.0 specification says

If a collapse clause is specified with a parameter value greater than 1, then the iterations of the associated loops to which the clause applies are collapsed into one larger iteration space ...

If you add the clause `collapse (2)` to a `#pragma omp parallel for` directive it will result in the following two loops being combined for parallelisation.

Question 5: Could you use the collapse clause with these loops and does it change your answer to the previous question?

- Lastly parallelise the `lissajous_multi.c` program using OpenMP and check that the results match the serial version.